AN

MINI PROJECT REPORT

ON

"Merge Sort vs Multithreaded Merge Sort"

Submitted in partial fulfillment of

FINAL YEAR OF ENGINEERING

In

COMPUTER ENGINEERING

Submitted By

Students Name	PRN No.
Satyam K. Gawali	**72328004F**
Aditya M. Pagare	**72328007L**
Sakshi G. Chaudhari	**72328001M**

Under guidance of

Prof. P. M. Achat



DEPARTMENT OF COMPUTER ENGINEERING

SHATABDI INSTITUTE OF ENGINEERING & RESEARCH

At post Agaskhind (Via Deolali camp Bhagur) Tal- Sinnar Dist- Nashik

Savitribai Phule Pune University, Pune

(A.Y. 2025-2026)

SHATABDI INSTITUTE OF ENGINEERING & RESEARCH AGASKHIND, TAL-SINNER, DIST-NASIK

DEPARTMENT OF COMPUTER ENGINEERING



CERTIFICATE

Students Name	PRN No.
Satyam K. Gawali	**72328004F**
Aditya M. Pagare	**72328007L**
Sakshi G. Chaudhari	**72328001M**

This is to certify that,

Has satisfactorily carried out and completed Mini Project Work entitled,

"Merge Sort vs Multithreaded Merge Sort"

It is submitted in the partial fulfillment of the prescribed syllabus of

Savitribai Phule Pune University, Pune

For the academic year 2025-2026

Prof. P. M. ACHAT
(Guide)

Prof. P. Y. SANVATSARKAR
HOD

Dr. S. P. SONAR
Vice Principal & Dean Academic

Dr. P. G. VISPUTE
Principal

ACKNOWLEDGEMENT

We would like to express my sincere gratitude to all those who have supported me throughout my training period and the preparation of this report.

Firstly, we would like to thank my project guide **Ms. P. M. Achat** for their invaluable gauidance, support,and encourangment throught the project.

We express our gratitude to **Ms. P. Y. Sanvatsarkar**, Assistant Professor and Head of Computer Engineering Department, for his constant encouragement, co-operation, valuable guidance and support. We express our sincere thanks to all Professors of department for their unfailing inspiration. Also, we would like to thank all the staff members of the department for their continuous support.

We would be failing in our duties if we do not make a mention of our family members including our parents for providing moral support, without which this work would not have been completed.

We express our gratitude to **Dr. S. P. Sonar** Vice Principal & Dean Academic, **Dr. P. G. Vispute,** Principal, of Shatabdi Institute of Engineering & Research, Nashik for their constant encouragement, co-operation, valuable guidance and support.

This kind of work cannot be finished without many others help, even some of them have not aware of their contribution and importance in producing this project. It is a great pleasure for us to take this opportunity to express our gratefulness to all of them.

Satyam K. Gawali

Aditya M. Pagare

Sakshi G. Chaudhari

DECLARATION

We are students of "Shatabdi Institute of Engineering & Research, Nashik" enrolled in the Degree in Bachelor of Engineering in Computer Engineering program, hereby declare that the mini project Report titled "Merge Sort vs Multithreaded Merge Sort" is our own original work.

We further declare that all references and sources used in the preparation of this report have been duly acknowledged. This report has not been submitted for any other academic purpose, nor has it been presented elsewhere for evaluation.

We express gratitude to supervisor **Prof. P. M. Achat** for their invaluable guidance and support during this training period.

Satyam K. Gawali

Aditya M. Pagare

Sakshi G. Chaudhari

ABSTRACT

Sorting is a key operation in computer science, forming the foundation for efficient data organization and retrieval. Merge Sort is a well-known divide-and-conquer algorithm that guarantees stable and predictable performance with a time complexity of *O(n log n)*. However, its traditional implementation executes sequentially, utilizing only a single processor core. To leverage the advantages of modern multi-core processors, this project implements a Multithreaded Merge Sort, which performs sorting tasks in parallel to reduce execution time.

The project compares the performance of standard Merge Sort and Multithreaded Merge Sort based on their execution time and complexity in both best and worst-case scenarios. Developed in Java, the implementation utilizes threads to execute merge operations concurrently, thereby improving efficiency through parallelism. The results demonstrate that while multithreading enhances performance on large datasets, it introduces additional overhead for smaller inputs. This study highlights the trade-off between computational parallelism and thread management, offering insights into optimizing sorting algorithms for modern multi-core systems.

CONTENT

Sr. No	Title Of Chapter	Page No
1	Aim	1
2	Objective	1
3	Problem Statement	1
4	Hardware and Software Requirements	1
5	Algorithm	2
6	Source Code	2
7	Sample Input / Output	4
8	Performance Analysis	4
9	Conclusion	4

1. Aim

Implement merge sort and multithreaded merge sort. Compare time required by both the algorithms. Also analyze the performance of each algorithm for the best case and the worst case.

2. Objectives

- To understand and implement the Merge Sort algorithm.
- To enhance Merge Sort using multithreading for parallel execution.
- To analyze and compare performance based on time and space complexity.
- To demonstrate the impact of parallelism on sorting performance.

3. Problem Statement

Sorting is a fundamental operation in computer science used in numerous applications. Traditional sorting algorithms like Merge Sort work sequentially and utilize only one CPU core. With the increasing availability of multi-core systems, parallel processing can significantly improve performance.

This project aims to implement **Multithreaded Merge Sort** to compare its efficiency with the standard Merge Sort in terms of execution time.

4. Hardware and Software Requirements

Hardware	Specifications
Processor	Intel i3/i5 or above
RAM	Minimum 4 GB
Storage	100 MB free space
System	Any multi-core processor

Software	Details
Operating System	Windows / Linux
Programming Language	Java
IDE	IntelliJ IDEA / VS Code / Eclipse
JDK Version	JDK 8 or above

5. Algorithm

A. Standard Merge Sort

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the two sorted halves into a single sorted array.

B. Multithreaded Merge Sort

1. Divide the array into two halves.
2. Create two threads to sort each half concurrently.
3. Wait for both threads to finish (join).
4. Merge the sorted halves.
5. For smaller subarrays, use normal merge sort to reduce overhead.

6. Source Code

```
import java.util.Arrays;
import java.util.Scanner;
class MergeSort {
    // Standard Merge Sort
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
    static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];
        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
}
// Multithreaded Merge Sort
class ThreadedMergeSort extends Thread {
    private int[] arr;
    private int left, right;
    private static final int THRESHOLD = 1000; // use normal merge sort for small
arrays
```

```
    ThreadedMergeSort(int[] arr, int left, int right) {
        this.arr = arr;
        this.left = left;
        this.right = right;
    }
    @Override
    public void run() {
        if (right - left < THRESHOLD) {
            MergeSort.mergeSort(arr, left, right);
        } else if (left < right) {
            int mid = left + (right - left) / 2;
            ThreadedMergeSort leftThread = new ThreadedMergeSort(arr, left, mid);
            ThreadedMergeSort rightThread = new ThreadedMergeSort(arr, mid + 1,
right);
            leftThread.start();
            rightThread.start();
            try {
                leftThread.join();
                rightThread.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            MergeSort.merge(arr, left, mid, right);
        }
    }
}
public class MergeSortComparison {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int[] arrCopy1 = Arrays.copyOf(arr, n);
        int[] arrCopy2 = Arrays.copyOf(arr, n);
        // Measure standard merge sort
        long startTime = System.nanoTime();
        MergeSort.mergeSort(arrCopy1, 0, n - 1);
        long endTime = System.nanoTime();
        double timeMergeSort = (endTime - startTime) / 1e6;
        // Measure multithreaded merge sort
        ThreadedMergeSort threadedSort = new ThreadedMergeSort(arrCopy2, 0, n - 1);
        long startThread = System.nanoTime();
        threadedSort.start();
        try { threadedSort.join();
} catch (InterruptedException e) { e.printStackTrace(); }
        long endThread = System.nanoTime();
        double timeThreaded = (endThread - startThread) / 1e6;
        System.out.println("\nSorted array (Merge Sort): " +
Arrays.toString(arrCopy1));
        System.out.println("Time taken (Merge Sort): " + timeMergeSort + " ms");
        System.out.println("\nSorted array (Multithreaded Merge Sort): " +
Arrays.toString(arrCopy2));
        System.out.println("Time taken (Multithreaded Merge Sort): " + timeThreaded +
" ms");
        sc.close();
    }
}
```

7. Sample Input / Output

Input:

```
Enter size of array: 6
Enter array elements:
10 7 8 9 1 5
```

Output:

```
Sorted array (Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Merge Sort): 0.947 ms

Sorted array (Multithreaded Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Multithreaded Merge Sort): 0.4146 ms
```

8. Performance Analysis

Parameter	Merge Sort	Multithreaded Merge Sort
Best Case	O(n log n)	O(n log n)
Worst Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Space Complexity	O(n)	O(n) + O(log n) (thread stack)

Observation:

- Both algorithms have the same theoretical complexity.
- Multithreading reduces actual runtime by executing parts of the algorithm in parallel.
- Speedup depends on the number of CPU cores and array size.
- For small arrays, thread creation overhead may reduce performance.

9. Conclusion

The project demonstrates that while both standard and multithreaded merge sort have the same time complexity, multithreading improves performance on large datasets through parallel execution. However, for small datasets, traditional merge sort may perform equally or better due to thread overhead.